

RAP 後端開發練習 5：Determinations (自動衍生欄位)

Lecture

這一課要解決什麼問題

rap03 的 `field ( readonly ) created_at, created_by;` 只做到一半：擋住了使用者自己填這兩個欄位，但沒有任何機制真的去填值——實際跑起來 `created_at/created_by` 會一直是初始值。**Determination (自動衍生欄位)** 就是 RAP 框架用來補上這一半的機制：在特定時機 (存檔時 / 資料變更當下) 自動觸發一段邏輯，把某些欄位的值算出來、寫回去，使用者完全不用管。

延續 rap03 的分工：

- **Part A：Managed Determination 語法**——這系統上可以編譯、可以啟用，但沒辦法真正執行 (Managed Runtime 白名單限制，見 rap03 Part C)；這次額外踩到一個這系統特有的語法差異 (見下方)，跟官方教材的落差比 rap03 遇到的更大
- **Part B：Unmanaged 沒有宣告式 Determination**——官方文件明講的限制，不是這系統的問題：邏輯要自己寫在 `CREATE/UPDATE` 方法裡，已用 `programrun` 完整驗證成功
- **Part C：對照總表**

Part A：Managed Determination 語法 (知識儲備，這系統無法執行)

在看完整範例之前，先認識會用到的語法元素 (查證官方 ABAP 語言文件 `ABENBDL_DETERMINATIONS / ABAPHANDLER_METH_DET` 確認，不是憑印象寫的)：

- `determination <名稱> on save { create; }`：寫在 BDEF 的 `{ }` 區塊裡，宣告一個 Determination ——<名稱> 自己取，`on save` (存檔時觸發，這一課用的時機) 或 `on modify` (資料一異動就觸發，比 `on save` 更早) 二選一；`{ create; }` 是觸發條件，可以是 `create/update/delete` 或 `field <欄位>` (欄位被改到才觸發)，這一課只在 `create` 時觸發。
- `managed implementation in class <類別> unique; ⚠` 一旦 BDEF 裡出現任何 Determination / Validation / Action，Managed BDEF 的 header 就不能再只寫單純的 `managed;` (rap03 教的寫法)，必須改成這個完整版本，指名「這個 BDEF 的自訂邏輯要去哪個類別找」——這跟 Unmanaged 的 `implementation unmanaged in class ... unique;` 語法很像，但關鍵字是 `managed` 不是 `unmanaged`，代表 CRUD 還是框架自動生成，只有 Determination 這類「額外邏輯」才需要你自己實作。

認識完語法後，這是這一課完整的 Managed BDEF (延伸 rap02/rap03 的 `ZI_RAP02_TASK`)：

```
managed implementation in class zbp_i_rap02_task unique;

define behavior for ZI_RAP02_TASK alias Task
persistent table zrap02_task
etag created_at
lock master
{
```

```

create;
update;
delete;

field ( mandatory ) description;

determination setCreationInfo on save { create; }
}

```

⚠ 這裡出現了 rap03 思考題 1 的答案，但也帶出一個新問題：`field ( readonly ) created_at, created_by;` 不見了——官方範例 (`ABENBDL_DETERMINATION_ABEXA` · `SoKey/AmountSum` 欄位) 明明是 `readonly` 欄位照樣可以被 `Determination` 寫入，這是官方教材的標準用法。但這系統的行為不一樣：底下會示範，這系統的 `obsolete Handler Method` 語法搭配 `readonly` 欄位，內部寫入會被直接擋下來，所以這一課的範例拿掉了這兩個欄位的 `readonly` (詳見下方「這一課踩到的語法坑」)。

Determination 的 Handler Method：這系統要用「舊式」語法

先看官方文件教的新式寫法 (`ABAPHANDLER_METH_DET`)：

```

METHODS det_on_save FOR DETERMINE ON SAVE
IMPORTING keys FOR bdef~det_save.

```

這段語法在這系統編譯失敗：`"DETERMINATION" expected, not "DETERMINE ON"`。——錯誤訊息直接告訴我們，這系統的編譯器要的是官方文件另外歸類成「**obsolete (過時)**」的舊式語法 (`ABAPMETHODS_FOR_DET_VAL_OBS : METHODS ... FOR DETERMINATION ...`)。跟 rap01 ~ rap04 遇過的模式一樣：這系統的 **RAP** 編譯器停在比官方目前主推教材更早的語言版本，新式語法反而編譯不過，舊式 (官方標成過時) 語法才是這系統認得的。

實測比對出這系統真正要的完整語法 (逐一測試錯誤訊息、一步步逼近出來的，不是文件裡找到的，過程見下方「這一課踩到的語法坑」)：

```

METHODS setCreationInfo FOR DETERMINATION Task~setCreationInfo
IMPORTING keys FOR Task.

```

跟官方新式語法對照：

官方新式 (這系統不支援)	這系統要的舊式寫法
<code>FOR DETERMINE ON SAVE</code>	<code>FOR DETERMINATION <alias>~<determination名稱></code> (存/改時機不重複寫， <code>BDEF</code> 那邊已經宣告過了)
<code>IMPORTING keys FOR bdef~det_save</code> (重複一次 <code>~det</code> 名稱)	<code>IMPORTING keys FOR <alias></code> (只要實體別名，不用重複 <code>Determination</code> 名稱)

認識完語法後，這是完整的 `Handler` 邏輯 (`ZBP_I_RAP02_TASK` 的 `Local Implementation Include`)：

```

CLASS lhc_task DEFINITION INHERITING FROM cl_abap_behavior_handler.
PRIVATE SECTION.

```

```

METHODS setCreationInfo FOR DETERMINATION Task~setCreationInfo
    IMPORTING keys FOR Task.
ENDCLASS.

CLASS lhc_task IMPLEMENTATION.

METHOD setCreationInfo.
    READ ENTITIES OF zi_rap02_task IN LOCAL MODE
        ENTITY task
            FIELDS ( created_at created_by ) WITH CORRESPONDING #( keys )
        RESULT DATA(tasks).

    MODIFY ENTITIES OF zi_rap02_task IN LOCAL MODE
        ENTITY task
            UPDATE FIELDS ( created_at created_by )
            WITH VALUE #( FOR ls_task IN tasks (
                %key          = ls_task-%key
                created_at = cl_abap_tstmp=>utclong2tstmp( utclong_current( ) )
                created_by = sy-uname ) ).
    ENDMETHOD.

ENDCLASS.

```

READ ENTITIES ... IN LOCAL MODE 跟 **MODIFY ENTITIES ... IN LOCAL MODE** 是 **Determination**

Handler 專屬的 **EML 變體**——**IN LOCAL MODE** 代表這次讀寫直接在 Transactional Buffer 裡操作，不會重新觸發整套 RAP 授權檢查 / Determination 鏈（避免自己觸發自己形成無窮迴圈）；**keys** 這個參數（前面的 **FOR Task** 幫你自動推導出型別）裝的是「這次被觸發 Determination 的所有實體 Key」，先用 **READ** 把要用到的欄位撈出來（即使只是要覆寫，也要先 **READ** 出當下的 Key 結構），再用 **MODIFY ... UPDATE FIELDS** 寫回新值。

這一課踩到的語法坑（逐步排錯，每一步都是實測錯誤訊息逼出來的）

1. **FOR DETERMINE ON SAVE** → **"DETERMINATION" expected, not "DETERMINE ON"**.——確認要改用 **obsolete** 語法 **FOR DETERMINATION**。
2. **FOR DETERMINATION ON SAVE IMPORTING keys FOR Task~setCreationInfo** → **A determination/validation is specified as "entity~name"**.——這個訊息容易誤導（看起來像在說格式錯誤，其實不是），真正原因是當時 **BDEF** 那邊的 **determination** 宣告根本還沒真的啟用成功（activation 一次打包 **BDEF+Class** 兩個物件，其中一個失敗會導致整批都不生效，即使沒報 **BDEF** 的錯）——教訓：**BDEF** 跟依賴它的 **Class** 要分開各自 **activate**，每次都讀回 **version=active** 確認真的生效，不能只看 **activation API** 有沒有回錯誤訊息。
3. 改對 **FOR DETERMINATION Task~setCreationInfo IMPORTING keys FOR Task~setCreationInfo** → **"TASK~SETCREATIONINFO" is not a subentity of the root entity**——**IMPORTING** 子句的 **FOR** 不能重複 **~detname**，只要實體別名。
4. 拿掉 **IMPORTING** 整段 → **"field FOR entity" or "IMPORTING field FOR" expected**——確認 **IMPORTING keys FOR Task**（只要別名）才是正確組合。

5. 邏輯本體 %tky → No component exists with the name "%TKY", but there is a component with a similar name "%KEY"——跟 rap03 讀取時踩過的坑一樣，這系統的 BDEF 衍生型別統一用 %key，不是官方教材常見的 %tky。
6. UPDATE FIELDS (created_at created_by) 寫入 field(readonly) 欄位 → The field "CREATED_AT" of entity "ZI_RAP02_TASK" cannot be modified.——⚠ 這是跟官方文件行為不一樣的地方：官方範例的 SoKey/AmountSum 都是 field(readonly) 還能被 Determination 寫入，但這系統用 obsolete FOR DETERMINATION 語法宣告的 Handler，IN LOCAL MODE 的寫入沒有拿到「內部框架寫入」的豁免，一樣被 readonly 擋下來。**Workaround** (這一課採用)：拿掉這兩個欄位的 field(readonly)——反正 Managed CUD 這系統本來就執行不了，拿掉純粹是為了讓語法範例能乾淨編譯，不影響教學重點。
7. created_at = utclong_current() → Result type of "UTCLONG_CURRENT" cannot be converted into the type of "CREATED_AT".——created_at 底層型別是 TIMESTAMPL (DEC21.7，rap02 就已經在用)，跟 utclong_current() 回傳的 utclong (8 byte 二進位) 是不同的底層表示法，這系統不支援兩者之間的隱含轉換，要用 cl_abap_tstmp=>utclong2tstmp(utclong_current()) 明確轉換。

全部修正後，ZI_RAP02_TASK (BDEF) + ZBP_I_RAP02_TASK (實作類別) 已經確認成功編譯啟用 (sap_inactive_objects 回 0 筆)，但一樣不要嘗試執行——Managed CUD 這系統在白名單限制 (rap03 Part C) 跟 Determination 語法本身無關，一樣會 Dump。

Part B：Unmanaged 沒有宣告式 Determination——這是官方明講的限制

先確認一件事：Unmanaged 沒辦法照抄 Part A 的 determination ... on save { } 語法，這不是這系統的限制，是 **RAP** 框架本身的設計。查證官方文件 ABENBDL_DETERMINATIONS 原文 (不是憑印象講的)：

Determinations are available for:

- Managed RAP BOs
- Unmanaged **and draft-enabled** RAP BOs
- **Caution: Not available for unmanaged, non-draft RAP BOs.**

ZI_RAP03_UMTEST 是 Unmanaged 且沒有啟用 **Draft** (Draft 是更進階的機制，這門課沒教)，剛好落在官方明講「不支援」的那一格——所以 rap03 思考題 / README 原本記錄的「Unmanaged 模式下要怎麼示範等效邏輯待查證」，答案是：沒有等效的宣告式語法，邏輯只能直接手寫在 **CREATE** (或 **UPDATE**) 方法裡，這一點程式碼上要接受，不用再找有沒有其他寫法。

等效寫法：在 **CREATE** 方法裡呼叫一個私有方法

沿用 rap03 的 ZRAP03_UMTEST (這次幫它加上 created_at/created_by 兩個欄位，欄位型別跟 rap02 一致：TIMESTAMPL / SYUNAME，都是標準 Data Element)。BDEF 一樣可以宣告 field (readonly) (Unmanaged 沒有 Determination 的 IN LOCAL MODE 內部寫入限制，readonly 只擋外部呼叫端，CREATE 方法內部用 Open SQL INSERT 完全不受影響)：

```
implementation unmanaged in class zbp_i_rap03_um4 unique;

define behavior for ZI_RAP03_UMTEST alias Test
lock master
{
```

```

create;

field ( readonly ) created_at, created_by;
}

```

設計想法：與其把「算出 created_at/created_by」的邏輯直接塞進 **create** 方法本體，不如拆成一個獨立的私有方法 (**determine_creation_info**) ——這個私有方法名稱故意取得像 **Determination**，因為它扮演的角色跟 Part A 的 **setCreationInfo** Handler Method 完全一樣（「算出這些欄位該填什麼值」），差別只在於**誰負責呼叫它**：Managed 世界是 RAP 框架在 **on save** 時機自動呼叫；Unmanaged 世界是你自己在 **create** 方法裡明確呼叫。這個對照，正是這一課想讓你體會的「宣告式 vs. 命令式」核心差異。

```

CLASS lcl_handler DEFINITION INHERITING FROM cl_abap_behavior_handler.
  PRIVATE SECTION.
    METHODS lock FOR LOCK
      IMPORTING it_lock FOR LOCK test.

    METHODS create FOR MODIFY
      IMPORTING it_create FOR CREATE test.

    METHODS read FOR READ
      IMPORTING it_read FOR READ test RESULT et_result.

    METHODS determine_creation_info
      RETURNING VALUE(rs_info) TYPE zrap03_umtest.
ENDCLASS.

CLASS lcl_handler IMPLEMENTATION.

  METHOD lock.
  ENDMETHOD.

  METHOD create.
    DATA(ls_info) = determine_creation_info( ).

    LOOP AT it_create INTO DATA(ls_create).
      INSERT zrap03_umtest FROM @( VALUE #(
        client      = sy-mandt
        id           = ls_create-id
        descr        = ls_create-descr
        created_at   = ls_info-created_at
        created_by   = ls_info-created_by ) ).
    ENDLOOP.
  ENDMETHOD.

  METHOD determine_creation_info.
    rs_info-created_at = cl_abap_tstmp=>utclong2tstmp( utclong_current( ) ).

```

```

rs_info-created_by = sy-uname.
ENDMETHOD.

METHOD read.
  LOOP AT it_read INTO DATA(ls_key).
    SELECT SINGLE id, descr, created_at, created_by
      FROM zrap03_umtest WHERE id = @ls_key-id INTO @DATA(ls_data).
    IF sy-subrc = 0.
      APPEND VALUE #(
        %key      = ls_key-%key
        id        = ls_data-id
        descr     = ls_data-descr
        created_at = ls_data-created_at
        created_by = ls_data-created_by ) TO et_result.
    ENDIF.
  ENDLOOP.
ENDMETHOD.

ENDCLASS.

```

幾個值得注意的細節：

- `determine_creation_info` 只呼叫一次 (`create` 方法最前面)，不是放在 `LOOP` 裡面——因為這一課的邏輯是「當下時間、當下使用者」，每一列都一樣，放迴圈外面省掉重複運算；如果你的邏輯是「每一列各自算不同的值」（例如逐列查關聯表），才需要放進迴圈內部各自呼叫。
- `RETURNING VALUE(rs_info) TYPE zrap03_umtest`——直接借用整張表的結構當回傳型別，圖個方便（只用得到 `created_at/created_by` 兩個欄位，其他欄位留空不影響），不是正式產品程式碼會採取的做法（正式場景通常會另外定義一個只含需要欄位的小結構），這裡是教學示範，優先求簡潔好懂。
- 這個私有方法沒有經過 **EML / RAP Runtime Framework**，就是一般 ABAP 方法呼叫——呼應「Unmanaged 沒有框架幫你分派 Determination」這件事：所有的「該在什麼時機呼叫什麼邏輯」，都要你自己在方法本體裡安排。

✅ 驗證結果 (`programrun` 無頭執行，完全成功)

```

REPORT zr_rap05_detdemo.

DATA(lv_test_id) = 'DET_TEST01'.

DELETE FROM zrap03_umtest WHERE id = @lv_test_id.

MODIFY ENTITIES OF zi_rap03_umtest
  ENTITY Test
    CREATE FIELDS ( id descr )
    WITH VALUE #( ( %cid = 'C1' id = lv_test_id descr = 'RAP05 Determination Demo' ) )
  FAILED DATA(ls_failed)
  REPORTED DATA(ls_reported).

```

```
COMMIT ENTITIES.
```

```
SELECT SINGLE id, descr, created_at, created_by
FROM zrap03_umtest
WHERE id = @lv_test_id
INTO @DATA(ls_check).
" ... 略：檢查 created_at/created_by 是否有值
```

輸出：

```
before EML
after EML, failed is initial: X
after commit entities
DB check OK, id = DET_TEST01
descr = RAP05 Determination Demo
created_at auto-filled: YES
created_by auto-filled correctly: MONICA
```

注意：EML **CREATE** 只傳了 **id** / **descr** 兩個欄位（完全沒提到 **created_at** / **created_by**），資料庫裡卻查得到這兩欄的真實值——這就是「自動衍生欄位」的效果，跟 Managed Determination 想達成的目的完全一樣，只是這裡是你自己在 **create** 方法裡手動接起來的，不是框架幫你接的。

Eclipse ADT：在既有 BDEF 裡加一個 Determination——Step by Step

rap03 已經教過怎麼從零建立 BDEF，這裡補上「怎麼在已經存在的 BDEF 裡加一個 Determination」（查證 SAP 官方 openSAP 《Building Apps with RAP》week3/unit5 ~ unit6 教材的實際操作步驟）：

1. Eclipse 打開目標 BDEF（例如 **ZI_RAP02_TASK**），在 **{ }** 區塊裡直接手打 **determination <名稱> on save { create; }** 這一行（沒有精靈能自動產生這一行，Determination 宣告要手寫）。
2. **Header** 那一行要記得跟著改：如果原本是單純 **managed;**（沒有實作類別），加了 Determination 之後必須改成 **managed implementation in class <類別名> unique;**——這一步很容易漏掉，漏了會在啟用時直接報錯要求指定實作類別。
3. 存檔（**Ctrl+S**）+ 啟用（**Activate**）。
4. 用 **Ctrl+1** 快速鍵生成 / 更新 **Handler Method** 骨架：游標點在 header 那一行的類別名稱上，按 **Ctrl+1**，選 **Create behavior implementation class**（如果類別還不存在）或直接進去手動加方法（如果類別已經存在，Eclipse 目前沒有針對「單獨補一個 Determination Handler Method」的快速修正，通常是手動在 Local Types Include 裡照著 Part A 的語法補上 **METHODS <名稱> FOR DETERMINATION ...** 宣告 + 對應的 **METHOD ... ENDMETHOD.** 實作）。
5. 存檔 + 啟用整個類別。

練習：幫自己的 BDEF 加一個 Determination

輪到你了，接續 rap03 練習做的物件：

① 必做（**Managed**，純語法練習）：對 rap03 練習 ① 建的 Managed BDEF（**ZI_RAP02_FLIGHT_PRACTICE** 那個），加一個 Determination——例如在 **create** 時自動把某個欄位設成固定值或系統時間（**SPFLI/SCARR** 沒

有 `created_at` 這種欄位，找一個你覺得適合的欄位示範語法即可，重點是練習 `determination ... on save { }` 語法 + `FOR DETERMINATION` Handler Method 怎麼寫，不用糾結欄位語意合不合理）。驗收依據：`checkruns / sap_inactive_objects` 確認啟用成功即可，不用（也不能）真的執行。

② 選做（**Unmanaged**，進階挑戰，真的能跑）：如果 rap03 練習 ② 有自己建 Unmanaged 物件，幫它的 `create` 方法加一個私有方法，自動填一個衍生欄位（可以是時間戳、可以是把兩個既有欄位組合成第三個欄位的值），照 Part B 的模式（私有方法 + 在 `create` 裡呼叫）改寫，最後用 EML 驗證資料真的自動填值成功。

完成後跟我說一下建立過程跟最終狀態，我會幫你核對。

Part C：Managed vs Unmanaged Determination 差異總表

	Managed	Unmanaged
宣告方式	BDEF 裡 <code>determination <名稱> on save/on modify { }</code> ，宣告式	沒有宣告，邏輯直接寫在 <code>CREATE/UPDATE</code> 方法本體（或呼叫的私有方法）裡
觸發時機	框架自動在 <code>on save / on modify</code> 時機呼叫	你自己決定什麼時候呼叫（通常就是在對應的 CUD 方法一開始）
底層 EML	<code>READ ENTITIES/MODIFY ENTITIES ... IN LOCAL MODE</code> （框架呼叫，開發者只寫方法本體）	不需要 EML，一般方法呼叫即可
這系統支援度	Draft 或非 Draft 都支援語法（但這系統 CUD 執行不了）	官方文件明講：非 Draft 完全不支援，這門課沒教 Draft
這系統能不能真正執行	❌（Managed Runtime 白名單限制，跟 Determination 語法無關）	✅ 已驗證成功
程式碼份量	少（一行宣告 + 一個 Handler Method）	中（私有方法 + 在 CRUD 方法裡明確呼叫）

學習目標

- 能寫出這系統適用的 Managed Determination 語法：`determination <名稱> on save { create; }`、BDEF header 改成 `managed implementation in class ... unique;`
- 能寫出這系統要求的 obsolete Handler Method 語法：`METHODS <名稱> FOR DETERMINATION <alias>~<det名稱> IMPORTING keys FOR <alias>`，知道跟官方新式 `FOR DETERMINE ON SAVE` 的差異
- 知道 `READ ENTITIES/MODIFY ENTITIES ... IN LOCAL MODE` 是 Determination Handler 專屬的 EML 變體，`%key` 是這系統的技術鍵欄位名稱
- 知道這系統這個語法組合下，`field(readonly)` 欄位無法被 Determination 內部寫入（跟官方範例行為不同），以及 `TIMESTAMPL` 需要 `cl_abap_tstmp=>utclong2tstmp()` 明確轉換
- 能說出官方文件對 Determination 可用範圍的明確限制：Managed 可以、Unmanaged 只有 Draft-enabled 才可以，非 Draft Unmanaged 完全不支援
- 能在 Unmanaged 實作類別裡設計「私有方法 + 在 CRUD 方法裡呼叫」這種等效 Determination 的手寫模式
- 能在 Eclipse ADT 對既有 BDEF 補上 Determination 宣告，知道要同步把 header 改成帶 `implementation in class` 的完整版本

物件清單

物件	名稱	型別	可執行性
Managed Behavior Definition (延伸)	ZI_RAP02_TASK	BDEF/BDO	語法正確，CUD 無法執行
Managed 實作類別	ZBP_I_RAP02_TASK	CLAS/OC	語法正確，無法執行
Managed EML 語法示範程式	ZR_RAP05_DEMO	PROG/P	語法正確，執行會 Dump
Unmanaged 測試表格 (延伸，加 created_at/created_by)	ZRAP03_UMTEST	TABL/DT	—
Unmanaged CDS View (延伸)	ZI_RAP03_UMTEST	DDLS/DF	—
Unmanaged Metadata Extension (延伸)	ZI_RAP03_UMTEST	DDLX/EX	—
Unmanaged Behavior Definition (延伸)	ZI_RAP03_UMTEST	BDEF/BDO	✓
Unmanaged 實作類別 (延伸，加 determine_creation_info)	ZBP_I_RAP03_UM4	CLAS/OC	✓
Unmanaged EML 驗證程式	ZR_RAP05_DETDEMO	PROG/P	✓ 已驗證成功

全部物件都在 \$TMP 套件，sap_inactive_objects 確認 0 筆殘留。

驗證方式

1. **Managed** 部分：checkruns / sap_inactive_objects 確認語法正確、成功啟用即可——不要嘗試執行 ZR_RAP05_DEMO，原因同 rap03 Part C，這是預期中的已知限制
2. **Unmanaged** 部分：已用 programrun 完整驗證成功，輸出見上方「驗證結果」，created_at / created_by 確認被正確自動填值

思考題

1. 這一課的 Managed Determination 用 on save (存檔時觸發)。如果改成 on modify (資料一異動就觸發)，對這個「填建立時間 / 建立人」的情境來說，用哪個時機比較合理？為什麼？
2. Unmanaged 版本的 determine_creation_info 完全沒有經過 RAP Framework，如果同一個 BDEF 除了 create 之外還有 update 操作，也想在 update 時自動更新一個「最後修改時間」欄位，程式碼要怎麼設計？(提示：想想這個私有方法是不是也能被 update 方法呼叫)
3. rap03 Part C 提到 Managed Runtime 白名單限制未來可能解除。如果真的解除了，這一課寫的 Managed Determination 語法 (determination ... on save { }、FOR DETERMINATION Handler Method) 要改多少才能執行？(提示：跟 rap03 思考題 3 一樣，語法本身可能完全不用改)

答案

Managed：zi_rap02_task.bdef.abap (延伸版)、zbp_i_rap02_task.clas.abap、zbp_i_rap02_task.clas.locals_imp.abap、zr_rap05_demo.prog.abap。**Unmanaged**：

zrap03_umtest.tabl.abap (延伸版) 、 zi_rap03_umtest.ddls.abap (延伸版) 、
zi_rap03_umtest.ddlx.abap (延伸版) 、 zi_rap03_umtest.bdef.abap (延伸版) 、
zbp_i_rap03_um4.clas.abap 、 zbp_i_rap03_um4.clas.locals_imp.abap (延伸版) 、
zr_rap05_detdemo.prog.abap (EML 驗證程式，已驗證執行成功) 。